4. Methods

Taxonomic binning strategy: Prior to model training, we discretized species-level relative abundances into a sample-by-species matrix A, where rows represent samples and columns correspond to microbial species. For each sample, species with non-zero abundance were sorted in descending order and divided evenly into B bins (we used $B = 100$). The most abundant N/B species (where N is the number of species with non-zero relative abundance in the sample) were assigned to bin B, the next N/B to bin $B - 1$, and so on. Species not detected in a given sample (those with zero relative abundance) were assigned a bin value of 0. For samples with fewer than B non-zero abundance species, species were distributed proportionally across bins 1 through B to ensure that low-diversity samples still spanned the full binned range. This avoided overconcentration of values in the highest bins, which could otherwise lead to underutilization of the lower bins during training and limit the model's ability to generalize across samples with varying species diversity.

Input embeddings and sample representation: For each MGX sample, we constructed an input sequence of microbial species paired with their binned abundance values.
Let A denote the sample-by-species matrix, where each entry $a_{ij} \in \{0,1,\ldots,100\}$ represents the binned abundance of species j in sample i. Each microbial species j was mapped to a learnable embedding vector via a `PyTorch` embedding layer, denoted $emb^{j}_{species}$. To embed the abundance bin values a_{ij}, we used a shallow MLP with ReLU activation, denoted $emb_{abundance}(a_{ij})$. The species and abundance embeddings were independently normalized using layer normalization. The final input token embedding for species j in sample i was computed as:

$$t_{ij} = \mathrm{LayerNorm}\left(emb^{j}_{species}\right) + \mathrm{LayerNomr}\left(emb_{abundance}\left(a_{ij}\right)\right). \quad (1)$$

To produce a sample-level representation for downstream prediction tasks, we prepended a special token <cls> to each sample sequence. This token had its own learnable species embedding $emb_s(< cls >)$ and was paired with an abundance bin value of 0. The resulting input embedding was defined as:

$$t_{i,<cls>} = \mathrm{LayerNorm}\left(emb^{<cls>}_{species}\right) + \mathrm{LayerNomr}\left(emb_{abundance}(0)\right). \quad (2)$$

The full sample sequence, consisting of the <cls> token followed by all microbial species with their binned abundances, $\{(< cls >,0), (\mathrm{species}_1, a_{1i}), (\mathrm{species}_2, a_{2i}), \ldots\}$, was used to

generate the corresponding embedding sequence $\{t_{i,<cls>}, t_{i1}, t_{i2}, \dots\}$, which was then passed as input to the transformer model.

Transformer configuration and masked pretraining procedure: GutBiomeGPT was implemented using the `TransformerEncoderLayer` module from `PyTorch`. The model architecture consisted of 8 stacked transformer layers, each containing 8 attention heads and a feedforward sublayer with a hidden dimension of 512. The input to the transformer was the embedded sequence for each sample, which included a special <cls> token followed by embeddings for each microbial species paired with its binned abundance value (see Equations 1 and 2).

Let $t_i^{(0)}$ denote the input embedding sequence for sample i. The transformer computed updated representations layer by layer as:

$$t_i^{(l)} = \text{TransformerBlock}\left(t_i^{(l-1)}\right), \qquad \text{for } l = 1,2,\dots,L. \tag{3}$$

After the final layer, the output at the <cls> position, $t_{i,<cls>}^{(L)}$, was extracted as the sample-level embedding. This embedding was passed to a task-specific prediction head during fine-tuning.

Pretraining was performed using a masked modeling objective. At each epoch, 25% of the non-zero-abundance species in each sample were randomly selected for masking, and the masked subset was selected independently across samples and epochs. Attention was computed only between unmasked species and between masked species and unmasked species; masked species could not attend to each other, but were allowed to attend to themselves.

After the transformer stack, each token's final-layer embedding is passed through a **three-layer MLP (hidden dimension = 512, ReLU activations)** that outputs one value per abundance bin. The model is trained to reconstruct the true bins of masked species by minimizing mean-squared error (MSE) between the MLP's predicted bin values and the ground-truth bins—computed only over masked positions and excluding zero-abundance species.

We used an embedding dimension of 512, 100 discrete abundance bins, and trained the model for 30 epochs on a single NVIDIA A100 GPU, accessed through a Google Colaboratory notebook, using a batch size of 64. The pretraining dataset (13,524 samples) was split 90–10 into training and validation subsets for convergence monitoring.

Embedding space visualization: We evaluated the structure of the pretrained <cls> embeddings using both visualization and clustering metrics. Two-dimensional projections were generated using `TSNE` from `sklearn.manifold`. Adjusted Rand Index was computed using `adjusted_rand_score` from `sklearn.metrics` after applying k-means clustering (`KMeans` from `sklearn.cluster, n_clusters`=34). Silhouette scores were computed with `silhouette_score` from `sklearn.metrics` using binary healthy vs. disease labels. Baseline embeddings included random vectors (`numpy.random.normal`) and PCA-derived embeddings (`PCA` from `sklearn.decomposition, n_components=512`) applied to the binned abundance matrix. All analyses were performed using `scikit-learn v1.3.0` and `NumPy v1.24.3`.

Embedding space visualization: We extracted the learned embedding vectors for all microbial species after pre-training and applied t-SNE to project them into two dimensions (`sklearn.manifold.TSNE, n_components=2, random_state=42`). To enhance interpretability, we limited the visualization to species belonging to the top 10 genera based on a composite score that combines both abundance and prevalence, defined as $ln(1 + 100 * mean_relative_abundance \times prevalence)$. For each genus, $mean_relative_abundance$ was computed using only the subset of samples in which the genus had non-zero abundance and $prevalence$ was defined as the fraction of samples in which the genus had non-zero abundance. Genus-level relative abundances were computed by summing the normalized read counts (relative abundances) of all species belonging to each genus within a sample. The 10 genera with the highest composite scores were selected, and all species from these genera were included in subsequent analyses.

To assess taxonomic structure, we applied k-means clustering (`KMeans` from `sklearn.cluster, n_clusters=10`) and computed the Adjusted Rand Index (ARI) between cluster assignments and true genus label using `adjusted_rand_score` from `sklearn.metrics`. We also evaluated pairwise cosine distances (`scipy.spatial.distance.cosine`) between species embeddings to compare intra-genus and inter-genus distances. As baselines, we generated random embeddings from a standard normal distribution (`numpy.random.normal`) and PCA-derived embeddings (`PCA` from `sklearn.decomposition, n_components=512`) applied to the species binned abundance matrix. All analyses were conducted using `scikit-learn v1.3.0, NumPy v1.24.3`, and `SciPy v1.11.1`.

Addressing class imbalance for fine-tunning tasks: In binary disease prediction tasks—including healthy vs. diseased, specific disease vs. healthy, and specific disease vs. all other diseases—we frequently encountered class imbalance. For instance, in the healthy vs. diseased task, the training set contained 7,992 healthy and 5,532 diseased samples. To

address this imbalance, we applied sample-level augmentation to the minority class using Gaussian noise injection. Specifically, for each minority-class sample, we generated an augmented sample by adding noise vector, $\epsilon \sim N(\mu, \sigma)$, drawn from a Gaussian distribution with mean $\mu = 0$ and standard deviation $\sigma = 0.1$. Noise was applied elementwise to only the non-zero entries of the sample abundance vector, preserving the original sparsity and compositional structure. The resulting augmented sample was then modified using the `clip()` function of `Numpy` in Python to replace negative values with 0. The final training set was constructed by combining the original majority-class samples, the minority-class samples, and the augmented minority-class set. The number of augmented samples was chosen such that the new size of the minority class matched the majority class.

This augmentation strategy was applied in both binary and multi-class settings. For multi-label classification, where the majority class was type 2 diabetes (T2D, n =– 936), all minority disease classes were upsampled to match this size.

Transfer learning for diseases with limitted training samples: For disease-type prediction tasks with limited sample sizes, we employed a transfer learning strategy to enhance model generalization. Diseases were grouped into nine clinically related categories (see **Supplementary File 1**), and the most prevalent disease in each group (denoted) was used to first train a model for that disease. For other diseases in the group, model weights were initialized from the trained model for the most prevalent disease rather than from the base pretrained backbone, allowing parameter reuse from phenotypically related tasks.

Feature selection: To further improve signal-to-noise ratio in binary disease-type classifiers, we applied feature selection using L1-regularized linear modeling, i.e., Least Absolute Shrinkage and Selection Operator (LASSO) regression. Specifically, we used `LassoCV` from `scikit-learn` to fit a logistic regression model on the training data and retained only microbial species with non-zero coefficients. This step was applied to the specific disease vs. healthy and specific disease vs. all-other classifiers, but not to the healthy vs. diseased task or to the multi-class setting, where broader taxonomic patterns were relevant.

Performance evaluation: Fine-tuning performance was evaluated using 10-fold cross-validation on the internal dataset (n = 13,524), with each fold consisting of a 90/10 training/validation split. External validation was performed using a held-out dataset of 927 samples from studies excluded during pretraining. For binary classification tasks (healthy vs. diseased, specific disease vs. healthy, and specific disease vs. all other diseases), we

report different sets of metrics depending on the evaluation setting. During cross-validation, since class imbalance was addressed through sample augmentation (see above), accuracy, F1-score, area under the receiver operating characteristic curve (AUROC), precision, and recall were calculated using the predicted vs. true labels for the positive class. For external validation, where class imbalance was preserved, both standard and macro-averaged metrics were reported. Macro metrics were computed by evaluating each class (positive and negative) independently and averaging the resulting scores, ensuring a balanced assessment that does not favor the majority class.

For the multi-class disease prediction task, which involves classification across 35 disease labels, class imbalance is intrinsic to both training and evaluation data. As such, we consistently report macro-averaged metrics—including macro F1-score, macro-AUCROC, macro precision, macro recall—for both cross-validation and external validation. These metrics provide a class-balanced summary of performance that reflects the model's ability to generalize across diverse clinical phenotypes.

All metrics were computed using the `torchmetrics` Python library (v1.3.0) of PyTorch. Specific functions included `Accuracy`, `F1Score`, `Precision`, `Recall`, and `AUROC`, with , `task="multiclass"` and `average= "macro"` were specified for each function for macro-averaged calculations.

Identification of highly attended species: To identify species with high model-assigned importance for each fine-tuning task, we computed the mean attention received by each microbial species from all other species. Let $Attn_{i,j'\to j}^{(hl)}$ denote the attention from species j' to species j in sample i (i.e., from token $t_{ij'}$ to token t_{ij}), head j, and layer l. For each species j, we define, $\overline{Attn}_{\to j}$, its aggregated attention score across N_j samples where it is present with non-zero abundance as:

$$\overline{Attn}_{\to j} = \frac{1}{64N_j} \sum_{i=1}^{N_j} \sum_{j' \in \{\text{Species} | j' \neq j\}} \sum_{l=1}^{8} \sum_{h=1}^{8} Attn_{i,j'\to j}^{(hl)}, \quad \forall j \in \{\text{Species}\}. \tag{4}$$

To facilitate interpretability and comparison across tasks, the scores were log-transformed using $Attn_{\to j}^{\log} = \ln\left(1 + \overline{Attn}_j\right)$ and scaled using robust min-max normalization applied independently for each fine-tunning task. Specifically, attention scores below the 5th percentile, P_5 were mapped to 0 and those above the 95th percentile, P_{i5}, to 1.

$$Attn_{\to j}^{norm} = \max\left(1, \min\left(0, \frac{Attn_{\to j}^{\log,} - P_5}{P_{95} - P_5}\right)\right), \quad \forall j \in \{\text{Species}\}. \tag{5}$$

The normalized attention scores, $Attn_j^{norm}$, ranges between 0 and 1. The top 100 species by normalized attention score were retained for downstream interpretation.

Co-attention analysis and clustering: To investigate species-species relationships learned by the model, we constructed a symmetric co-attention matrix, where each entry jj' represents the average bidirectional attention between species j and j', computed by mean attention across all samples, heads, and layers:

$$Attn_{(j,j')} = \frac{\left(\overline{Attn}_{j\to j'} + \overline{Attn}_{j'\to j}\right)}{2}, \quad \forall j, j' \in \{\text{Species} | j \neq j''\}, \tag{6}$$

where, $\overline{Attn}_{j\to j'}$ (and $\overline{Attn}_{j'\to j}$) represent the mean attention from species j to species j' (species j' to species j) across heads, layers, and samples where both species j and j' are present (with non-zero abundance).

Differential abundance, enrichment and attention-abudnacne correlation analyses
Differential abundance (DA) analysis was conducted for each binary fine-tuning task—healthy vs. diseased and specific disease vs. healthy—using ANCOM-BC [10] v1.4.0 (via R), on the species-level relative-abundance data. Significant results were reported for q < 0.05 (Benjamini–Hochberg FDR). We then ranked all species by their mean attention score from the final GutBiomeGPT model trained on the full 13,524-sample training dataset and designated the top K = 100 species as the "highly attended" set. For each task, we constructed a 2×2 contingency table contrasting species in the top-K set vs. those outside it against DA-significant vs. non-DA species and applied Fisher's exact test (`scipy.stats.fisher_exact`). P-values across tasks were adjusted for multiple comparisons using the Benjamini–Hochberg procedure (`statsmodels.stats.multitest.multipletests, method='fdr_bh'`). Enrichment was expressed as the percentage of top-K species that were DA-significant: 100 ×(number of DA-significant species in top-K highly-attended set)/K.

To assess whether higher attention aligns with larger average fold-changes, we computed $\log_2$ fold-change for every species per task as $\log_2\left(\frac{\bar{x}_{disease}+10^{-6}}{\bar{x}_{healthy}+10^{-6}}\right)$, where $\bar{x}_{helathy}$ and $\bar{x}_{disease}$ are mean relative abundance within healthy and disease groups and 10^{-6} is to prevent division by zero. We then calculated Spearman's rank correlation (`scipy.stats.spearmanr`) between each species' mean attention and absolute value of $\log_2$ fold-change across all taxa. Resulting p-values were likewise FDR-corrected across fine-tunning tasks.

Clustering of co-attending microbial species: To identify clusters of co-attending microbial species, we applied two complementary clustering approaches to the symmetric

co-attention matrix derived from our transformer model: hierarchical clustering and k-means clustering following dimensionality reduction via UMAP. For hierarchical clustering, we first converted the co-attention matrix into a distance matrix by using the inverse of the mean co-attention scores. We then applied the `scipy.cluster.hierarchy.linkage` function (with the `Ward` method) and used `scipy.cluster.hierarchy.dendrogram` to compute and visualize the species ordering. To avoid nested structures inherent to dendrograms and facilitate interpretation, we converted the hierarchical tree into discrete clusters using `scipy.cluster.hierarchy.fcluster` with a fixed number of clusters.

For the second approach, we first embedded species into a two-dimensional space using Uniform Manifold Approximation and Projection (UMAP), as implemented in the `umap-learn` package (`umap.UMAP` class with default parameters), applied directly to the same distance matrix. We then performed k-means clustering on the resulting UMAP embeddings using `sklearn.cluster.KMeans`. We varied the number of clusters and selected a value that provided clear visual separation in the UMAP plot while maintaining biological plausibility.

References